Operating Systems 2

Exercise Sheet N°2

(*Synchronization*)

Part I (Parallelism - Cont.)

Exercise N°1 :

1. In the UNIX system, does every process have a parent? What happens when a process becomes an orphan (its parent dies)? When does a process become a zombie?
2. To launch several processes in parallel for the same application, you can use either the system calls `fork()` or `pthread_create()`. Which one would you choose and why?
3. In UNIX, process creation is achieved through duplication.
   a. Give one advantage and one disadvantage.
   b. Give the advantages of lightweight processes (threads) compared to processes.

Exercise N°2 :

Consider a file named `OS2`. To speed up the search for the word `Fork` in the OS2 file, the main process creates four child processes. Each child process searches in one of the four parts of the file by calling the following function: :

`int Search (char * File, char * Word, int Part)` where :

- `File` is the name of the file, i.e., `OS2`,
- `Word` is the target word, i.e., `Fork`, and
- `Part` is part 1, 2, 3, or 4 of the file.

This function returns **1** on success and **0** otherwise.

After returning from the Search function, the child process sends the result to the parent process using the exit system call (`exit(0)` for success, `exit(1)` for failure). When the parent process learns that one of its children succeeded, it kills the remaining child processes.

1. Write a `C` program that performs the above behavior.

Note: *Do not write the code for the Search function.*

Part II (Mutual Exclusion)

Exercise N°3 :

1. Why does data sharing cause problems in a multiprogrammed time-sharing system ? Does UNIX provide mechanisms to control access to shared data ? What is a critical section ?

2. There are several solutions for mutual exclusion, such as **interrupt masking** and the use of the assembler instruction **Test and Set (TAS)**. List the disadvantages of both solutions.

3. A hardware solution for mutual exclusion consists of using an indivisible instruction called `SWAP`, which swaps the values of two variables as follows:

```
void SWAP (var a, b : boolean)
var temp: boolean;
Debut
 temp = a;
 a= b;
 b = temp;
Fin.
```

   a. Propose a solution to the mutual exclusion problem for two processes using this instruction.

Exercise N°4 :

1. **Alternation** : A first attempt at mutual exclusion uses a common variable "`turn`" initialized to 0, meaning : `turn = i` : `Process` P_i is allowed to execute its critical section.
   The structure of a process P_i with i=0,1 is :

```
// Shared data
turn = 0 ;
Pi
Repeat
 While (turn ≠ i) ;
 <Critical Section>
 turn = 1-i ;
Until false
```

   a. Propose comments for this solution.

2. Another solution improves the previous one by using a `boolean array flag[0,1] = {false, false}`, where each element indicates whether a process is in its critical section (`flag[i] = true` means P_i is executing its critical section).
   The structure of a process P_i with i=0,1 is :

```
// Shared data
flag [0, 1] = {false, false} ;
Pi
Repeat
 While (flag[1-i]) ;
 flag[i]= vrai ;
 <Critical Section>
 flag[i]= false ;
Until false
```

   a. Comment on this solution.

3. This second solution was improved. Now, `flag[i] = true` means Pi wants to enter or is in the critical section.

```
//Shared data
flag [0, 1] = {false, false} ;
Pi
Repeat
 flag[i]= true ;
 While (flag[1-i]) ;
 <Critical Section>
 flag[i]= false ;
Until false
```

   a. What are your comments ?

4. **Dekker's Solution**: Dekker's algorithm uses two shared variables : a `boolean array flag[0,1] = {false, false}` and a variable "`turn`" that can take values `0` or `1`.

```
//Shared data
flag [0, 1] = {false, false} ;
turn=0 ;
Pi
Repeat
 flag[i]= true ;
 While (flag[1-i])
  If turn = 1-i Then
    flag[i]= false ;
    While (turn ≠ i) ;
    flag[i]= true ;
  EnfIf
 EndWhile
 <Critical Section>
 turn = 1-i; flag[i]= false ;
Until false
```

   a. Does this solution ensure mutual exclusion? What are your comments?

5. **Peterson's Solution**: This solution generalizes **Dekker's** algorithm.

```
//Shared data
interested [0, 1] = {false, false} ;
turn=0 ;
Pi
Repeat
 interested [i]= true ; // wants to enter critical section
 turn = i ;
 While (turn == i et interested [1-i]) ;
<Critical Section>
 interested [i]= false ;
Until false
```

   a. What are your comments?

Exercise N°5 :

Assume in Unix we can define variables x and y shared by two processes as :

```
shared long x = 0 ;
shared long y = 0 ;
```

Two processes execute the following code:

```
Process P1
Begin
x = x + 1;
y = y + 1;
printf("x=%d,y=%d\n", x, y);
End
```

```
Process P2
Begin
x = x * 2;
y = y * 2;
printf("x=%d,y=%d\n", x, y);
End
```

a. Using a semaphore, modify the code so that printf always displays identical values for x and y.

Exercise N°6 :

1. Three concurrent processes P1, P2, and P3 share the variables n and out. To control access to these shared variables, a programmer proposes the following code:

```
mutex1, mutex2 : Semaphore init(1) ;
```

```
Process P1
Begin
P(mutex1) ; P(mutex2) ;
out=out+1 ; n=n-1 ;
V(mutex2) ; V(mutex1) ;
End
```

```
Process P2
Begin
P(mutex2) ;
out=out-1 ;
V(mutex2) ;
End
```

```
Process P3
Begin
P(mutex1) ;
n=n+1 ;
V(mutex1) ;
End
```

a. Is this solution correct? If not, identify which of the four required mutual exclusion conditions (i.e. **Dijkstra properties**) are not satisfied. Propose a correct solution.

2. We want to compute in parallel the product of two matrices A and B of order n (n×n). We create m (m<n) threads, each computing several rows of the result matrix R :

$$pour \quad j = 0 \,à\, n-1 \quad R[i,j] = \sum_{k=0,n-1} A[i,k] * B[k,j]$$

a. Using semaphore operations P and V, give the code for the processes: ComputeRows(). Indicate the semaphores and shared variables used.